

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии  
Департамент цифровых, робототехнических систем и электроники

**ОТЧЕТ**  
**ПО ЛАБОРАТОРНОЙ РАБОТЕ №4**  
**дисциплины**  
**«Основы нейронных сетей»**  
**Вариант 10**

Выполнил:  
Рябинин Егор Алексеевич  
3 курс, группа ИВТ-б-о-23-2,  
09.03.01 «Информатика и  
вычислительная техника»,  
направленность (профиль)  
«Программное обеспечение средств  
вычислительной техники и  
автоматизированных систем», очная  
форма обучения

---

(подпись)

Проверил:  
Доцент департамента цифровых,  
робототехнических систем и  
электроники института перспективной  
инженерии  
Воронкин Роман Александрович

---

(подпись)

Отчет защищен с оценкой \_\_\_\_\_ Дата защиты \_\_\_\_\_

Ставрополь, 2026 г

**Тема:** применение методов глубокого обучения для анализа и классификации текстовых данных.

**Цель:** изучить основные подходы к обработке естественного языка с использованием нейронных сетей: токенизацию, создание последовательностей, методы Bag of Words и Embedding, а также освоить построение архитектур полносвязных сетей с регуляризацией (Dropout, BatchNormalization) для задач многоклассовой и бинарной классификации текстов на примере различных датасетов.

**Порядок выполнения работы:**

Ссылка на репозиторий:

[https://github.com/bohemiaaaaa/Lab4\\_Neural-networks](https://github.com/bohemiaaaaa/Lab4_Neural-networks)

**Pytorch:**

**Задание 1.** В домашнем задании Lite вам предлагается поработать подробнее с параметрами словаря и формированием гиперпараметров нейронной сети. Создайте 9 нейросетей с различными гиперпараметрами (см. пункт 2 и 3)

Для этого необходимо:

1. Воссоздать ноутбук, аналогичный ноутбуку практической части №1, загрузив при этом необходимую нам базу (код уже доступен в ноутбуке).

2. Задать в ноутбуке следующие параметры для размера словаря, ширины окна и шага:

– Размер словаря - от 10000 до 20000 (выбрать меньшее значение диапазона, если будет перегрузка ОЗУ и перезапуск подключения к Colaboratory)

– Ширина окна - от 1000 до 2000

– Шаг - от 100 до 500 (на обучение лучше влияет наименьший шаг, но это может перегрузить ОЗУ).

3. Создать архитектуру сети и задать гиперпараметры. Можно воспользоваться шаблоном:

- Добавьте модель прямого распространения Sequential()
- Добавьте один или несколько полносвязных (Dense) слоёв
- Добавьте слои Dropout() и BatchNormalization()
- Добавьте выходной полносвязный слой с количеством нейронов, соответствующим количеству классов (число писателей)

Напомним, что точность сети можно проверить по значению показателя val\_accuracy на конце каждой эпохи.

```
# Импорт библиотек
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import gdown
import os
import re
import matplotlib.pyplot as plt
```

Рисунок 1 – Импорт библиотек

```
# Загрузка и распаковка датасета
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l7/writers.zip', None, quiet=True)
!unzip -qo writers.zip -d writers/
```

Рисунок 2 – Загрузка и распаковка датасета

```

# Чтение и подготовка текстов
FILE_DIR = 'writers'
SIG_TRAIN = 'обучающая'
SIG_TEST = 'тестовая'

CLASS_LIST = []
text_train = []
text_test = []

for file_name in os.listdir(FILE_DIR):
    m = re.match('\((.+)\) (\S+)_', file_name)
    if m:
        class_name = m[1]
        subset_name = m[2].lower()
        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name
        if is_train or is_test:
            if class_name not in CLASS_LIST:
                CLASS_LIST.append(class_name)
                text_train.append('')
                text_test.append('')
            cls = CLASS_LIST.index(class_name)
            with open(f'{FILE_DIR}/{file_name}', 'r') as f:
                text = f.read()
            subset = text_train if is_train else text_test
            subset[cls] += ' ' + text.replace('\n', ' ')

CLASS_COUNT = len(CLASS_LIST)
VOCAB_SIZE = 15000
print(f"Классы: {CLASS_LIST}")
print(f"Количество классов: {CLASS_COUNT}")

```

Рисунок 3 – Чтение текстов писателей и формирование списка классов

```

# Токенизация и построение словаря
tokenizer = Tokenizer(num_words=VOCAB_SIZE,
                      filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~«»\t\n\ха0\ufe0f',
                      lower=True, split=' ', oov_token='неизвестное_слово')
tokenizer.fit_on_texts(text_train)

seq_train = tokenizer.texts_to_sequences(text_train)
seq_test = tokenizer.texts_to_sequences(text_test)

```

Рисунок 4 – Токенизация текстов и преобразование в последовательности индексов

```

# Функция разбиения последовательности на отрезки
def split_sequence(sequence, win_size, hop):
    if len(sequence) < win_size:
        return []
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]

# Функция формирования выборок
def vectorize_sequence(seq_list, win_size, hop):
    x_list = []
    y_list = []
    for cls, seq in enumerate(seq_list):
        chunks = split_sequence(seq, win_size, hop)
        for ch in chunks:
            x_list.append(ch)
            y_list.append(cls)
    x_pad = pad_sequences(x_list, maxlen=win_size, padding='post', truncating='post')
    return torch.tensor(x_pad, dtype=torch.long), torch.tensor(y_list, dtype=torch.long)

```

Рисунок 5 – Функции разбиения на фрагменты и создания модели

```

class TextClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes, win_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.fc1 = nn.Linear(win_size * embed_dim, 256)
        self.bn1 = nn.BatchNorm1d(256)
        self.fc2 = nn.Linear(256, 128)
        self.bn2 = nn.BatchNorm1d(128)
        self.fc3 = nn.Linear(128, num_classes)
        self.dropout = nn.Dropout(0.3)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.embedding(x)
        x = x.view(x.size(0), -1)
        x = self.relu(self.bn1(self.fc1(x)))
        x = self.dropout(x)
        x = self.relu(self.bn2(self.fc2(x)))
        x = self.dropout(x)
        return self.fc3(x)

```

Рисунок 6 – Создание модели

```

# Функция обучения модели
def train_model(model, train_loader, val_loader, epochs=10, lr=0.001):
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

    val_accs = []

    for epoch in range(epochs):
        model.train()
        for x_batch, y_batch in train_loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            loss = criterion(model(x_batch), y_batch)
            loss.backward()
            optimizer.step()

        scheduler.step()

        model.eval()
        correct, total = 0, 0
        with torch.no_grad():
            for x_batch, y_batch in val_loader:
                x_batch, y_batch = x_batch.to(device), y_batch.to(device)
                outputs = model(x_batch)
                _, predicted = torch.max(outputs, 1)
                total += y_batch.size(0)
                correct += (predicted == y_batch).sum().item()

        val_acc = correct / total
        val_accs.append(val_acc)
        print(f"Epoch {epoch+1}/{epochs} - val_accuracy: {val_acc:.4f}")

    return max(val_accs)

```

Рисунок 7 – Функция обучения модели

```

# Эксперимент 1: WIN_SIZE=1000, HOP=100
print("\n=== ЭКСПЕРИМЕНТ 1: WIN_SIZE=1000, HOP=100 ===")

WIN_SIZE = 1000
WIN_HOP = 100

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc1 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 1: Лучшая val_accuracy = {acc1:.4f}")

=== ЭКСПЕРИМЕНТ 1: WIN_SIZE=1000, HOP=100 ===
Epoch 1/10 - val_accuracy: 0.3889
Epoch 2/10 - val_accuracy: 0.4777
Epoch 3/10 - val_accuracy: 0.5021
Epoch 4/10 - val_accuracy: 0.4874
Epoch 5/10 - val_accuracy: 0.5036
Epoch 6/10 - val_accuracy: 0.5115
Epoch 7/10 - val_accuracy: 0.5165
Epoch 8/10 - val_accuracy: 0.5121
Epoch 9/10 - val_accuracy: 0.5070
Epoch 10/10 - val_accuracy: 0.5132
Результат 1: Лучшая val_accuracy = 0.5132

```

Рисунок 8 – Эксперимент 1: WIN\_SIZE=1000, WIN\_HOP=100

```

# Эксперимент 2: WIN_SIZE=1000, HOP=300
print("\n=== ЭКСПЕРИМЕНТ 2: WIN_SIZE=1000, HOP=300 ===")

WIN_SIZE = 1000
WIN_HOP = 300

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc2 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 2: Лучшая val_accuracy = {acc2:.4f}")

=== ЭКСПЕРИМЕНТ 2: WIN_SIZE=1000, HOP=300 ===
Epoch 1/10 - val_accuracy: 0.3478
Epoch 2/10 - val_accuracy: 0.3411
Epoch 3/10 - val_accuracy: 0.3572
Epoch 4/10 - val_accuracy: 0.3550
Epoch 5/10 - val_accuracy: 0.3720
Epoch 6/10 - val_accuracy: 0.3707
Epoch 7/10 - val_accuracy: 0.3702
Epoch 8/10 - val_accuracy: 0.3684
Epoch 9/10 - val_accuracy: 0.3805
Epoch 10/10 - val_accuracy: 0.3734
Результат 2: Лучшая val_accuracy = 0.3734

```

Рисунок 9 – Эксперимент 2: WIN\_SIZE=1000, WIN\_HOP=300

```

# Эксперимент 3: WIN_SIZE=1000, HOP=500
print("\n=== ЭКСПЕРИМЕНТ 3: WIN_SIZE=1000, HOP=500 ===")

WIN_SIZE = 1000
WIN_HOP = 500

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc3 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 3: Лучшая val_accuracy = {acc3:.4f}")

=== ЭКСПЕРИМЕНТ 3: WIN_SIZE=1000, HOP=500 ===
Epoch 1/10 - val_accuracy: 0.3022
Epoch 2/10 - val_accuracy: 0.3306
Epoch 3/10 - val_accuracy: 0.3194
Epoch 4/10 - val_accuracy: 0.3097
Epoch 5/10 - val_accuracy: 0.3194
Epoch 6/10 - val_accuracy: 0.3201
Epoch 7/10 - val_accuracy: 0.3351
Epoch 8/10 - val_accuracy: 0.3187
Epoch 9/10 - val_accuracy: 0.3284
Epoch 10/10 - val_accuracy: 0.3291
Результат 3: Лучшая val_accuracy = 0.3291

```

Рисунок 10 – Эксперимент 3: WIN\_SIZE=1000, WIN\_HOP=500

```

# Эксперимент 4: WIN_SIZE=1500, HOP=100
print("\n=== ЭКСПЕРИМЕНТ 4: WIN_SIZE=1500, HOP=100 ===")

WIN_SIZE = 1500
WIN_HOP = 100

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc4 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 4: Лучшая val_accuracy = {acc4:.4f}")

=== ЭКСПЕРИМЕНТ 4: WIN_SIZE=1500, HOP=100 ===
Epoch 1/10 - val_accuracy: 0.4093
Epoch 2/10 - val_accuracy: 0.5165
Epoch 3/10 - val_accuracy: 0.5219
Epoch 4/10 - val_accuracy: 0.5370
Epoch 5/10 - val_accuracy: 0.5406
Epoch 6/10 - val_accuracy: 0.5452
Epoch 7/10 - val_accuracy: 0.5409
Epoch 8/10 - val_accuracy: 0.5473
Epoch 9/10 - val_accuracy: 0.5377
Epoch 10/10 - val_accuracy: 0.5466
Результат 4: Лучшая val_accuracy = 0.5466

```

Рисунок 11 – Эксперимент 4: WIN\_SIZE=1500, WIN\_HOP=100

```
# Эксперимент 5: WIN_SIZE=1500, HOP=300
print("\n=== ЭКСПЕРИМЕНТ 5: WIN_SIZE=1500, HOP=300 ===")

WIN_SIZE = 1500
WIN_HOP = 300

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc5 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 5: Лучшая val_accuracy = {acc5:.4f}")
```

---

```
=== ЭКСПЕРИМЕНТ 5: WIN_SIZE=1500, HOP=300 ===
Epoch 1/10 - val_accuracy: 0.3306
Epoch 2/10 - val_accuracy: 0.3568
Epoch 3/10 - val_accuracy: 0.3680
Epoch 4/10 - val_accuracy: 0.3806
Epoch 5/10 - val_accuracy: 0.3860
Epoch 6/10 - val_accuracy: 0.3928
Epoch 7/10 - val_accuracy: 0.3883
Epoch 8/10 - val_accuracy: 0.3842
Epoch 9/10 - val_accuracy: 0.3892
Epoch 10/10 - val_accuracy: 0.3941
Результат 5: Лучшая val_accuracy = 0.3941
```

Рисунок 12 – Эксперимент 5: WIN\_SIZE=1500, WIN\_HOP=300

```
# Эксперимент 6: WIN_SIZE=1500, HOP=500
print("\n=== ЭКСПЕРИМЕНТ 6: WIN_SIZE=1500, HOP=500 ===")

WIN_SIZE = 1500
WIN_HOP = 500

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc6 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 6: Лучшая val_accuracy = {acc6:.4f}")
```

---

```
=== ЭКСПЕРИМЕНТ 6: WIN_SIZE=1500, HOP=500 ===
Epoch 1/10 - val_accuracy: 0.2856
Epoch 2/10 - val_accuracy: 0.3411
Epoch 3/10 - val_accuracy: 0.3508
Epoch 4/10 - val_accuracy: 0.3673
Epoch 5/10 - val_accuracy: 0.3598
Epoch 6/10 - val_accuracy: 0.3523
Epoch 7/10 - val_accuracy: 0.3463
Epoch 8/10 - val_accuracy: 0.3658
Epoch 9/10 - val_accuracy: 0.3673
Epoch 10/10 - val_accuracy: 0.3508
Результат 6: Лучшая val accuracy = 0.3508
```

Рисунок 13 – Эксперимент 6: WIN\_SIZE=1500, WIN\_HOP=500

```

# Эксперимент 7: WIN_SIZE=2000, HOP=100
print("\n=== ЭКСПЕРИМЕНТ 7: WIN_SIZE=2000, HOP=100 ===")

WIN_SIZE = 2000
WIN_HOP = 100

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc7 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 7: Лучшая val_accuracy = {acc7:.4f}")

=== ЭКСПЕРИМЕНТ 7: WIN_SIZE=2000, HOP=100 ===
Epoch 1/10 - val_accuracy: 0.4940
Epoch 2/10 - val_accuracy: 0.5477
Epoch 3/10 - val_accuracy: 0.5478
Epoch 4/10 - val_accuracy: 0.5676
Epoch 5/10 - val_accuracy: 0.5528
Epoch 6/10 - val_accuracy: 0.5555
Epoch 7/10 - val_accuracy: 0.5605
Epoch 8/10 - val_accuracy: 0.5626
Epoch 9/10 - val_accuracy: 0.5557
Epoch 10/10 - val_accuracy: 0.5696
Результат 7: Лучшая val_accuracy = 0.5696

```

Рисунок 14 – Эксперимент 7: WIN\_SIZE=2000, WIN\_HOP=100

```

# Эксперимент 8: WIN_SIZE=2000, HOP=300
print("\n=== ЭКСПЕРИМЕНТ 8: WIN_SIZE=2000, HOP=300 ===")

WIN_SIZE = 2000
WIN_HOP = 300

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc8 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 8: Лучшая val_accuracy = {acc8:.4f}")

=== ЭКСПЕРИМЕНТ 8: WIN_SIZE=2000, HOP=300 ===
Epoch 1/10 - val_accuracy: 0.3668
Epoch 2/10 - val_accuracy: 0.3650
Epoch 3/10 - val_accuracy: 0.4016
Epoch 4/10 - val_accuracy: 0.3962
Epoch 5/10 - val_accuracy: 0.4003
Epoch 6/10 - val_accuracy: 0.4034
Epoch 7/10 - val_accuracy: 0.4116
Epoch 8/10 - val_accuracy: 0.3980
Epoch 9/10 - val_accuracy: 0.4034
Epoch 10/10 - val_accuracy: 0.4066
Результат 8: Лучшая val_accuracy = 0.4066

```

Рисунок 15 – Эксперимент 8: WIN\_SIZE=2000, WIN\_HOP=300

```
# Эксперимент 9: WIN_SIZE=2000, HOP=500
print("\n=== ЭКСПЕРИМЕНТ 9: WIN_SIZE=2000, HOP=500 ===")

WIN_SIZE = 2000
WIN_HOP = 500

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=128, shuffle=False)

model = TextClassifier(VOCAB_SIZE, 128, CLASS_COUNT, WIN_SIZE)
acc9 = compile_and_train(model, train_loader, test_loader, epochs=10)

print(f"Результат 9: Лучшая val_accuracy = {acc9:.4f}")
```

```
=== ЭКСПЕРИМЕНТ 9: WIN_SIZE=2000, HOP=500 ===
Epoch 1/10 - val_accuracy: 0.3268
Epoch 2/10 - val_accuracy: 0.3720
Epoch 3/10 - val_accuracy: 0.3788
Epoch 4/10 - val_accuracy: 0.3803
Epoch 5/10 - val_accuracy: 0.3765
Epoch 6/10 - val_accuracy: 0.3810
Epoch 7/10 - val_accuracy: 0.3886
Epoch 8/10 - val_accuracy: 0.3916
Epoch 9/10 - val_accuracy: 0.3976
Epoch 10/10 - val_accuracy: 0.3938
Результат 9: Лучшая val_accuracy = 0.3938
```

Рисунок 16 – Эксперимент 9: WIN\_SIZE=2000, WIN\_HOP=500

| Эксперимент | WIN_SIZE | WIN_HOP | Лучшая val_accuracy |
|-------------|----------|---------|---------------------|
| 1           | 1000     | 100     | 0.5132              |
| 2           | 1000     | 300     | 0.3734              |
| 3           | 1000     | 500     | 0.3291              |
| 4           | 1500     | 100     | 0.5466              |
| 5           | 1500     | 300     | 0.3941              |
| 6           | 1500     | 500     | 0.3508              |
| 7           | 2000     | 100     | 0.5696              |
| 8           | 2000     | 300     | 0.4066              |
| 9           | 2000     | 500     | 0.3938              |

Лучший результат: 0.5696 (56.96%)  
 Лучшие параметры: WIN\_SIZE=2000, WIN\_HOP=100

Рисунок 17 – Сравнение результатов

## **Tensorflow:**

**Задание 1.** В домашнем задании Lite вам предлагается поработать подробнее с параметрами словаря и формированием гиперпараметров нейронной сети. Создайте 9 нейросетей с различными гиперпараметрами (см. пункт 2 и 3)

Для этого необходимо:

1. Воссоздать ноутбук, аналогичный ноутбуку практической части №1, загрузив при этом необходимую нам базу (код уже доступен в ноутбуке).

2. Задать в ноутбуке следующие параметры для размера словаря, ширины окна и шага:

- Размер словаря - от 10000 до 20000 (выбрать меньшее значение диапазона, если будет перегрузка ОЗУ и перезапуск подключения к Colaboratory)

- Ширина окна - от 1000 до 2000

- Шаг - от 100 до 500 (на обучение лучше влияет наименьший шаг, но это может перегрузить ОЗУ).

3. Создать архитектуру сети и задать гиперпараметры. Можно воспользоваться шаблоном:

- Добавьте модель прямого распространения Sequential()

- Добавьте один или несколько полносвязных (Dense) слоёв

- Добавьте слои Dropout() и BatchNormalization()

- Добавьте выходной полносвязный слой с количеством нейронов, соответствующим количеству классов (число писателей)

Напомним, что точность сети можно проверить по значению показателя val\_acc на конце каждой эпохи.

```

# 1. Импорт библиотек
import numpy as np
from tensorflow.keras import utils
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Flatten, Embedding
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import gdown
import os
import re
import matplotlib.pyplot as plt

# 2. Загрузка и распаковка датасета
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/writers.zip', None, quiet=True)
!unzip -qo writers.zip -d writers/

# 3. Константы и загрузка текстов
FILE_DIR = 'writers'
SIG_TRAIN = 'обучающая'
SIG_TEST = 'тестовая'

CLASS_LIST = []
text_train = []
text_test = []

for file_name in os.listdir(FILE_DIR):
    m = re.match('\((.+)\) (\S+)', file_name)
    if m:
        class_name = m[1]
        subset_name = m[2].lower()
        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name
        if is_train or is_test:
            if class_name not in CLASS_LIST:
                CLASS_LIST.append(class_name)
                text_train.append('')
                text_test.append('')
            cls = CLASS_LIST.index(class_name)
            with open(f'{FILE_DIR}/{file_name}', 'r') as f:
                text = f.read()
            subset = text_train if is_train else text_test
            subset[cls] += ' ' + text.replace('\n', ' ')

CLASS_COUNT = len(CLASS_LIST)
print(f"Классы: {CLASS_LIST}")

```

Рисунок 18 – Импорт библиотек, загрузка датасета, подготовка текстов и токенизация



```

print("\n=== ЭКСПЕРИМЕНТ 1: WIN_SIZE=1000, HOP=100 ===")

WIN_SIZE = 1000
WIN_HOP = 100

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc1 = max(history.history['val_accuracy'])
print(f"Результат 1: Лучшая val_accuracy = {acc1:.4f}")

```

```

=== ЭКСПЕРИМЕНТ 1: WIN_SIZE=1000, HOP=100 ===
Размер выборки: Train (17640, 1000), Test (6686, 1000)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated
  warnings.warn(
Epoch 1/10
138/138 ————— 13s 51ms/step - accuracy: 0.7618 - loss: 0.6851 - val_accuracy: 0.6279 - val_loss: 1.1436
Epoch 2/10
138/138 ————— 3s 20ms/step - accuracy: 0.9980 - loss: 0.0212 - val_accuracy: 0.7300 - val_loss: 0.8306
Epoch 3/10
138/138 ————— 3s 20ms/step - accuracy: 0.9997 - loss: 0.0059 - val_accuracy: 0.7459 - val_loss: 0.7968
Epoch 4/10
138/138 ————— 3s 23ms/step - accuracy: 0.9999 - loss: 0.0033 - val_accuracy: 0.7549 - val_loss: 0.7879
Epoch 5/10
138/138 ————— 3s 20ms/step - accuracy: 0.9999 - loss: 0.0020 - val_accuracy: 0.7550 - val_loss: 0.7968
Epoch 6/10
138/138 ————— 3s 20ms/step - accuracy: 0.9999 - loss: 0.0015 - val_accuracy: 0.7653 - val_loss: 0.7767
Epoch 7/10
138/138 ————— 3s 20ms/step - accuracy: 1.0000 - loss: 0.0013 - val_accuracy: 0.7556 - val_loss: 0.8206
Epoch 8/10
138/138 ————— 3s 20ms/step - accuracy: 0.9999 - loss: 0.0010 - val_accuracy: 0.7710 - val_loss: 0.7876
Epoch 9/10
138/138 ————— 3s 20ms/step - accuracy: 1.0000 - loss: 7.7226e-04 - val_accuracy: 0.7582 - val_loss: 0.8152
Epoch 10/10
138/138 ————— 3s 19ms/step - accuracy: 1.0000 - loss: 6.4064e-04 - val_accuracy: 0.7975 - val_loss: 0.7192
Результат 1: Лучшая val_accuracy = 0.7975

```

Рисунок 21 – Эксперимент 1: WIN\_SIZE=1000, WIN\_HOP=100

```

print("\n=== ЭКСПЕРИМЕНТ 2: WIN_SIZE=1000, HOP=300 ===")

WIN_SIZE = 1000
WIN_HOP = 300

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc2 = max(history.history['val_accuracy'])
print(f"Результат 2: Лучшая val_accuracy = {acc2:.4f}")

```

```

=== ЭКСПЕРИМЕНТ 2: WIN_SIZE=1000, HOP=300 ===
Размер выборки: Train (5883, 1000), Test (2231, 1000)
Epoch 1/10
46/46 ————— 10s 106ms/step - accuracy: 0.4746 - loss: 1.5119 - val_accuracy: 0.3102 - val_loss: 1.6084
Epoch 2/10
46/46 ————— 1s 19ms/step - accuracy: 0.9371 - loss: 0.2890 - val_accuracy: 0.4827 - val_loss: 1.2980
Epoch 3/10
46/46 ————— 1s 19ms/step - accuracy: 0.9966 - loss: 0.0542 - val_accuracy: 0.5235 - val_loss: 1.2060
Epoch 4/10
46/46 ————— 1s 19ms/step - accuracy: 0.9993 - loss: 0.0228 - val_accuracy: 0.5742 - val_loss: 1.1577
Epoch 5/10
46/46 ————— 1s 19ms/step - accuracy: 0.9998 - loss: 0.0141 - val_accuracy: 0.5903 - val_loss: 1.1021
Epoch 6/10
46/46 ————— 1s 20ms/step - accuracy: 0.9993 - loss: 0.0097 - val_accuracy: 0.6091 - val_loss: 1.0969
Epoch 7/10
46/46 ————— 1s 19ms/step - accuracy: 0.9998 - loss: 0.0074 - val_accuracy: 0.6369 - val_loss: 1.0410
Epoch 8/10
46/46 ————— 1s 19ms/step - accuracy: 0.9998 - loss: 0.0052 - val_accuracy: 0.6432 - val_loss: 1.0366
Epoch 9/10
46/46 ————— 1s 21ms/step - accuracy: 1.0000 - loss: 0.0042 - val_accuracy: 0.6513 - val_loss: 1.0286
Epoch 10/10
46/46 ————— 1s 21ms/step - accuracy: 1.0000 - loss: 0.0035 - val_accuracy: 0.6602 - val_loss: 1.0194
Результат 2: Лучшая val_accuracy = 0.6602

```

Рисунок 22 – Эксперимент 2: WIN\_SIZE=1000, WIN\_HOP=300

```

print("\n=== ЭКСПЕРИМЕНТ 3: WIN_SIZE=1000, HOP=500 ===")

WIN_SIZE = 1000
WIN_HOP = 500

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc3 = max(history.history['val_accuracy'])
print(f"Результат 3: Лучшая val_accuracy = {acc3:.4f}")

```

```

=== ЭКСПЕРИМЕНТ 3: WIN_SIZE=1000, HOP=500 ===
Размер выборки: Train (3531, 1000), Test (1340, 1000)
Epoch 1/10
28/28 ----- 10s 157ms/step - accuracy: 0.3829 - loss: 1.7837 - val_accuracy: 0.3022 - val_loss: 1.8214
Epoch 2/10
28/28 ----- 4s 22ms/step - accuracy: 0.8318 - loss: 0.5554 - val_accuracy: 0.4045 - val_loss: 1.4542
Epoch 3/10
28/28 ----- 1s 20ms/step - accuracy: 0.9626 - loss: 0.1905 - val_accuracy: 0.4037 - val_loss: 1.4104
Epoch 4/10
28/28 ----- 1s 20ms/step - accuracy: 0.9926 - loss: 0.0727 - val_accuracy: 0.4246 - val_loss: 1.3734
Epoch 5/10
28/28 ----- 1s 25ms/step - accuracy: 0.9975 - loss: 0.0359 - val_accuracy: 0.4037 - val_loss: 1.4156
Epoch 6/10
28/28 ----- 1s 22ms/step - accuracy: 0.9983 - loss: 0.0241 - val_accuracy: 0.4127 - val_loss: 1.4163
Epoch 7/10
28/28 ----- 1s 23ms/step - accuracy: 0.9997 - loss: 0.0180 - val_accuracy: 0.4440 - val_loss: 1.3850
Epoch 8/10
28/28 ----- 1s 20ms/step - accuracy: 0.9997 - loss: 0.0124 - val_accuracy: 0.4552 - val_loss: 1.4035
Epoch 9/10
28/28 ----- 1s 20ms/step - accuracy: 0.9992 - loss: 0.0108 - val_accuracy: 0.4649 - val_loss: 1.3755
Epoch 10/10
28/28 ----- 1s 21ms/step - accuracy: 0.9997 - loss: 0.0081 - val_accuracy: 0.4619 - val_loss: 1.4161
Результат 3: Лучшая val_accuracy = 0.4649

```

Рисунок 23 – Эксперимент 3: WIN\_SIZE=1000, WIN\_HOP=500

```

print("\n=== ЭКСПЕРИМЕНТ 4: WIN_SIZE=1500, HOP=100 ===")

WIN_SIZE = 1500
WIN_HOP = 100

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc4 = max(history.history['val_accuracy'])
print(f"Результат 4: Лучшая val_accuracy = {acc4:.4f}")

```

```

=== ЭКСПЕРИМЕНТ 4: WIN_SIZE=1500, HOP=100 ===
Размер выборки: Train (17610, 1500), Test (6656, 1500)
Epoch 1/10
138/138 ----- 15s 53ms/step - accuracy: 0.8287 - loss: 0.5142 - val_accuracy: 0.6980 - val_loss: 0.9649
Epoch 2/10
138/138 ----- 4s 29ms/step - accuracy: 0.9985 - loss: 0.0148 - val_accuracy: 0.7809 - val_loss: 0.6721
Epoch 3/10
138/138 ----- 4s 29ms/step - accuracy: 0.9997 - loss: 0.0046 - val_accuracy: 0.7931 - val_loss: 0.6123
Epoch 4/10
138/138 ----- 4s 30ms/step - accuracy: 0.9998 - loss: 0.0028 - val_accuracy: 0.7828 - val_loss: 0.6490
Epoch 5/10
138/138 ----- 4s 30ms/step - accuracy: 1.0000 - loss: 0.0016 - val_accuracy: 0.8048 - val_loss: 0.6158
Epoch 6/10
138/138 ----- 4s 29ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.7942 - val_loss: 0.6366
Epoch 7/10
138/138 ----- 4s 29ms/step - accuracy: 0.9999 - loss: 0.0011 - val_accuracy: 0.8012 - val_loss: 0.6286
Epoch 8/10
138/138 ----- 4s 29ms/step - accuracy: 1.0000 - loss: 6.8148e-04 - val_accuracy: 0.8131 - val_loss: 0.6029
Epoch 9/10
138/138 ----- 4s 28ms/step - accuracy: 1.0000 - loss: 5.9618e-04 - val_accuracy: 0.8042 - val_loss: 0.6182
Epoch 10/10
138/138 ----- 4s 27ms/step - accuracy: 1.0000 - loss: 4.9283e-04 - val_accuracy: 0.8120 - val_loss: 0.5896
Результат 4: Лучшая val_accuracy = 0.8131

```

Рисунок 24 – Эксперимент 4: WIN\_SIZE=1500, WIN\_HOP=100

```

print("\n=== ЭКСПЕРИМЕНТ 5: WIN_SIZE=1500, HOP=300 ===")

WIN_SIZE = 1500
WIN_HOP = 300

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc5 = max(history.history['val_accuracy'])
print(f"Результат 5: Лучшая val_accuracy = {acc5:.4f}")

=== ЭКСПЕРИМЕНТ 5: WIN_SIZE=1500, HOP=300 ===
Размер выборки: Train (5872, 1500), Test (2220, 1500)
Epoch 1/10
46/46 ————— 12s 140ms/step - accuracy: 0.5293 - loss: 1.3456 - val_accuracy: 0.3685 - val_loss: 1.5369
Epoch 2/10
46/46 ————— 1s 27ms/step - accuracy: 0.9670 - loss: 0.1894 - val_accuracy: 0.5486 - val_loss: 1.2310
Epoch 3/10
46/46 ————— 1s 27ms/step - accuracy: 0.9978 - loss: 0.0330 - val_accuracy: 0.6486 - val_loss: 1.0504
Epoch 4/10
46/46 ————— 1s 27ms/step - accuracy: 0.9993 - loss: 0.0152 - val_accuracy: 0.6820 - val_loss: 0.9832
Epoch 5/10
46/46 ————— 1s 28ms/step - accuracy: 0.9990 - loss: 0.0109 - val_accuracy: 0.6851 - val_loss: 0.9733
Epoch 6/10
46/46 ————— 1s 27ms/step - accuracy: 0.9997 - loss: 0.0075 - val_accuracy: 0.7104 - val_loss: 0.8862
Epoch 7/10
46/46 ————— 3s 32ms/step - accuracy: 0.9998 - loss: 0.0056 - val_accuracy: 0.7077 - val_loss: 0.9242
Epoch 8/10
46/46 ————— 2s 33ms/step - accuracy: 1.0000 - loss: 0.0040 - val_accuracy: 0.7099 - val_loss: 0.9017
Epoch 9/10
46/46 ————— 1s 28ms/step - accuracy: 1.0000 - loss: 0.0032 - val_accuracy: 0.7306 - val_loss: 0.8429
Epoch 10/10
46/46 ————— 1s 28ms/step - accuracy: 1.0000 - loss: 0.0032 - val_accuracy: 0.7212 - val_loss: 0.8625
Результат 5: Лучшая val_accuracy = 0.7306

```

Рисунок 25 – Эксперимент 5: WIN\_SIZE=1500, WIN\_HOP=300

```

print("\n=== ЭКСПЕРИМЕНТ 6: WIN_SIZE=1500, HOP=500 ===")

WIN_SIZE = 1500
WIN_HOP = 500

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc6 = max(history.history['val_accuracy'])
print(f"Результат 6: Лучшая val_accuracy = {acc6:.4f}")

=== ЭКСПЕРИМЕНТ 6: WIN_SIZE=1500, HOP=500 ===
Размер выборки: Train (3525, 1500), Test (1334, 1500)
Epoch 1/10
28/28 ————— 11s 180ms/step - accuracy: 0.4113 - loss: 1.7044 - val_accuracy: 0.3388 - val_loss: 1.8138
Epoch 2/10
28/28 ————— 1s 29ms/step - accuracy: 0.8769 - loss: 0.4810 - val_accuracy: 0.4198 - val_loss: 1.4606
Epoch 3/10
28/28 ————— 1s 29ms/step - accuracy: 0.9818 - loss: 0.1372 - val_accuracy: 0.4723 - val_loss: 1.3525
Epoch 4/10
28/28 ————— 1s 29ms/step - accuracy: 0.9952 - loss: 0.0554 - val_accuracy: 0.4783 - val_loss: 1.3440
Epoch 5/10
28/28 ————— 1s 31ms/step - accuracy: 0.9983 - loss: 0.0315 - val_accuracy: 0.4865 - val_loss: 1.3422
Epoch 6/10
28/28 ————— 1s 31ms/step - accuracy: 0.9983 - loss: 0.0203 - val_accuracy: 0.4985 - val_loss: 1.3518
Epoch 7/10
28/28 ————— 1s 32ms/step - accuracy: 0.9989 - loss: 0.0159 - val_accuracy: 0.5075 - val_loss: 1.3161
Epoch 8/10
28/28 ————— 1s 30ms/step - accuracy: 1.0000 - loss: 0.0104 - val_accuracy: 0.5210 - val_loss: 1.2815
Epoch 9/10
28/28 ————— 1s 30ms/step - accuracy: 0.9994 - loss: 0.0107 - val_accuracy: 0.5367 - val_loss: 1.2438
Epoch 10/10
28/28 ————— 1s 30ms/step - accuracy: 0.9994 - loss: 0.0088 - val_accuracy: 0.5435 - val_loss: 1.2410
Результат 6: Лучшая val_accuracy = 0.5435

```

Рисунок 26 – Эксперимент 6: WIN\_SIZE=1500, WIN\_HOP=500

```

print("\n=== ЭКСПЕРИМЕНТ 7: WIN_SIZE=2000, HOP=100 ===")

WIN_SIZE = 2000
WIN_HOP = 100

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc7 = max(history.history['val_accuracy'])
print(f"Результат 7: Лучшая val_accuracy = {acc7:.4f}")

```

```

=== ЭКСПЕРИМЕНТ 7: WIN_SIZE=2000, HOP=100 ===
Размер выборки: Train (17580, 2000), Test (6626, 2000)
Epoch 1/10
138/138 ————— 19s 72ms/step - accuracy: 0.8449 - loss: 0.4546 - val_accuracy: 0.7025 - val_loss: 0.9682
Epoch 2/10
138/138 ————— 5s 38ms/step - accuracy: 0.9990 - loss: 0.0139 - val_accuracy: 0.7791 - val_loss: 0.7183
Epoch 3/10
138/138 ————— 5s 39ms/step - accuracy: 0.9998 - loss: 0.0042 - val_accuracy: 0.7999 - val_loss: 0.6863
Epoch 4/10
138/138 ————— 5s 38ms/step - accuracy: 0.9998 - loss: 0.0023 - val_accuracy: 0.8006 - val_loss: 0.6677
Epoch 5/10
138/138 ————— 5s 38ms/step - accuracy: 1.0000 - loss: 0.0016 - val_accuracy: 0.8038 - val_loss: 0.6503
Epoch 6/10
138/138 ————— 5s 36ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.8204 - val_loss: 0.6363
Epoch 7/10
138/138 ————— 5s 36ms/step - accuracy: 1.0000 - loss: 8.2204e-04 - val_accuracy: 0.8189 - val_loss: 0.6157
Epoch 8/10
138/138 ————— 5s 36ms/step - accuracy: 1.0000 - loss: 6.5911e-04 - val_accuracy: 0.8248 - val_loss: 0.5957
Epoch 9/10
138/138 ————— 5s 36ms/step - accuracy: 1.0000 - loss: 5.3208e-04 - val_accuracy: 0.8136 - val_loss: 0.6478
Epoch 10/10
138/138 ————— 6s 42ms/step - accuracy: 1.0000 - loss: 4.3268e-04 - val_accuracy: 0.8192 - val_loss: 0.6369
Результат 7: Лучшая val_accuracy = 0.8248

```

Рисунок 27 – Эксперимент 7: WIN\_SIZE=2000, WIN\_HOP=100

```

WIN_SIZE = 2000
WIN_HOP = 300

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc8 = max(history.history['val_accuracy'])
print(f"Результат 8: Лучшая val_accuracy = {acc8:.4f}")

```

```

=== ЭКСПЕРИМЕНТ 8: WIN_SIZE=2000, HOP=300 ===
Размер выборки: Train (5861, 2000), Test (2211, 2000)
Epoch 1/10
46/46 ————— 14s 139ms/step - accuracy: 0.5755 - loss: 1.2168 - val_accuracy: 0.6196 - val_loss: 1.3388
Epoch 2/10
46/46 ————— 2s 36ms/step - accuracy: 0.9797 - loss: 0.1388 - val_accuracy: 0.7038 - val_loss: 0.9358
Epoch 3/10
46/46 ————— 2s 36ms/step - accuracy: 0.9978 - loss: 0.0293 - val_accuracy: 0.7422 - val_loss: 0.8142
Epoch 4/10
46/46 ————— 2s 38ms/step - accuracy: 0.9991 - loss: 0.0135 - val_accuracy: 0.7463 - val_loss: 0.7844
Epoch 5/10
46/46 ————— 2s 38ms/step - accuracy: 0.9998 - loss: 0.0092 - val_accuracy: 0.7603 - val_loss: 0.7739
Epoch 6/10
46/46 ————— 2s 37ms/step - accuracy: 0.9998 - loss: 0.0065 - val_accuracy: 0.7499 - val_loss: 0.7959
Epoch 7/10
46/46 ————— 2s 38ms/step - accuracy: 0.9998 - loss: 0.0051 - val_accuracy: 0.7607 - val_loss: 0.7648
Epoch 8/10
46/46 ————— 2s 38ms/step - accuracy: 0.9998 - loss: 0.0042 - val_accuracy: 0.7589 - val_loss: 0.7496
Epoch 9/10
46/46 ————— 2s 38ms/step - accuracy: 0.9995 - loss: 0.0039 - val_accuracy: 0.7350 - val_loss: 0.8218
Epoch 10/10
46/46 ————— 2s 38ms/step - accuracy: 0.9998 - loss: 0.0027 - val_accuracy: 0.7508 - val_loss: 0.7789
Результат 8: Лучшая val_accuracy = 0.7607

```

Рисунок 28 – Эксперимент 8: WIN\_SIZE=2000, WIN\_HOP=300

```

WIN_SIZE = 2000
WIN_HOP = 500

x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
print(f"Размер выборки: Train {x_train.shape}, Test {x_test.shape}")

model = create_model(WIN_SIZE, VOCAB_SIZE, CLASS_COUNT)
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_data=(x_test, y_test), verbose=1)

acc9 = max(history.history['val_accuracy'])
print(f"Результат 9: Лучшая val_accuracy = {acc9:.4f}")

=== ЭКСПЕРИМЕНТ 9: WIN_SIZE=2000, HOP=500 ===
Размер выборки: Train (3519, 2000), Test (1328, 2000)
Epoch 1/10
28/28 ----- 13s 229ms/step - accuracy: 0.4297 - loss: 1.6044 - val_accuracy: 0.3426 - val_loss: 1.6515
Epoch 2/10
28/28 ----- 1s 39ms/step - accuracy: 0.9048 - loss: 0.4395 - val_accuracy: 0.4217 - val_loss: 1.4649
Epoch 3/10
28/28 ----- 1s 39ms/step - accuracy: 0.9841 - loss: 0.1300 - val_accuracy: 0.4616 - val_loss: 1.3761
Epoch 4/10
28/28 ----- 1s 39ms/step - accuracy: 0.9946 - loss: 0.0551 - val_accuracy: 0.5038 - val_loss: 1.2782
Epoch 5/10
28/28 ----- 1s 40ms/step - accuracy: 0.9977 - loss: 0.0282 - val_accuracy: 0.5226 - val_loss: 1.2497
Epoch 6/10
28/28 ----- 1s 40ms/step - accuracy: 0.9989 - loss: 0.0202 - val_accuracy: 0.5459 - val_loss: 1.1803
Epoch 7/10
28/28 ----- 1s 40ms/step - accuracy: 0.9991 - loss: 0.0134 - val_accuracy: 0.5633 - val_loss: 1.1885
Epoch 8/10
28/28 ----- 1s 40ms/step - accuracy: 0.9994 - loss: 0.0117 - val_accuracy: 0.5723 - val_loss: 1.1800
Epoch 9/10
28/28 ----- 1s 41ms/step - accuracy: 0.9997 - loss: 0.0081 - val_accuracy: 0.5625 - val_loss: 1.2092
Epoch 10/10
28/28 ----- 1s 43ms/step - accuracy: 1.0000 - loss: 0.0070 - val_accuracy: 0.5791 - val_loss: 1.1676
Результат 9: Лучшая val_accuracy = 0.5791

```

Рисунок 29 – Эксперимент 9: WIN\_SIZE=2000, WIN\_HOP=500

| Эксперимент | WIN_SIZE | WIN_HOP | Лучшая val_аccuracy |
|-------------|----------|---------|---------------------|
| 1           | 1000     | 100     | 0.7975              |
| 2           | 1000     | 300     | 0.6602              |
| 3           | 1000     | 500     | 0.4649              |
| 4           | 1500     | 100     | 0.8131              |
| 5           | 1500     | 300     | 0.7306              |
| 6           | 1500     | 500     | 0.5435              |
| 7           | 2000     | 100     | 0.8248              |
| 8           | 2000     | 300     | 0.7607              |
| 9           | 2000     | 500     | 0.5791              |

Лучший результат: 0.8248 (82.48%)  
 Лучшие параметры: WIN\_SIZE=2000, WIN\_HOP=100

Рисунок 30 – Сравнение результатов

**Задание 2.** В данном домашнем задании предоставляется возможность поработать с задачей по распознаванию позитивных и негативных отзывы людей по автомобилю Tesla. База для обучения содержит два текстовых файла с рядом строчных отзывов с мнением людей об автомобиле Tesla, соответственно негативного и позитивного содержания. Ссылка на скачивание базы уже включена в ноутбук задания.

Необходимо выполнить следующие действия:

1. Загрузите саму базу по ссылке и подготовьте файлы базы для обработки.
2. Создайте обучающую и проверочную выборки, обратив особое внимание на балансировку базы: количество примеров каждого класса должно быть примерно одного порядка.
3. Подготовьте выборки для обучения и обучите сеть. Добейтесь результата точности сети в 85-90% на проверочной выборке.

```
# Работа с массивами данных
import numpy as np

# Работа с таблицами
import pandas as pd

# Отрисовка графиков
import matplotlib.pyplot as plt

# Функции-утилиты для работы с категориальными данными
from tensorflow.keras import utils

# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation

# Токенизатор для преобразование текстов в последовательности
from tensorflow.keras.preprocessing.text import Tokenizer

# Заполнение последовательностей до определенной длины
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Загрузка датасетов из облака google
import gdown

# Для работы с файлами в Colaboratory
import os

# Отрисовка графиков
import matplotlib.pyplot as plt

%matplotlib inline
```

Рисунок 31 – Импорт библиотек

```
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/tesla.zip', None, quiet=True)
```

Рисунок 32 – Скачивание датасета

```
# Распаковка архива в папку writers
!unzip -qo tesla.zip -d tesla/

# Просмотр содержимого папки
!ls tesla
```

Рисунок 33 – Распаковка архива и просмотр файлов

```
# Объявляем функции для чтения файла. На вход отправляем путь к файлу
def read_text(file_name):

    # Задаем открытие нужного файла в режиме чтения
    read_file = open(file_name, 'r')

    # Читаем текст
    text = read_file.read()

    # Переносы строки переводим в пробелы
    text = text.replace("\n", " ")

    # Возвращаем текст файла
    return text

# Объявляем интересующие нас классы
class_names = ["Негативный отзыв", "Позитивный отзыв"]

# Считаем количество классов
num_classes = len(class_names)
```

Рисунок 34 – Функция чтения текстовых файлов и объявление классов

```
import os
# Создаём список под тексты для обучающей выборки
texts_list = []

# Циклом проводим итерацию по текстовым файлам в папке отзывов
for j in os.listdir('/content/tesla/'):

    # Добавляем каждый файл в общий список для выборки
    texts_list.append(read_text('/content/tesla/' + j))

    # Выводим на экран сообщение о добавлении файла
    print(j, 'добавлен в обучающую выборку')
```

Рисунок 35 – Загрузка отзывов в список texts\_list

```

# Узнаем объём каждого текста в словах и символах
texts_len = [len(text) for text in texts_list]

# Устанавливаем "счётчик" номера текста
t_num = 0

# Выводим на экран информационное сообщение
print(f'Размеры текстов по порядку (в токенах):')

# Циклом проводим итерацию по списку с объёмами текстов
for text_len in texts_len:

    # Запускаем "счётчик" номера текста
    t_num += 1

    # Выводим на экран сообщение о номере и объёме текста
    print(f'Текст №{t_num}: {text_len}')

```

Размеры текстов по порядку (в токенах):  
Текст №1: 134535  
Текст №2: 213381

Рисунок 36 – Вывод размеров загруженных текстов в символах

```

# Создаём список с вложенным циклом по длинам текстов, где i - 100% текста, i/5 - 20% текста
train_len_shares = [(i - round(i/5)) for i in texts_len]

# Устанавливаем "счётчик" номера текста
t_num = 0

# Циклом проводим итерацию по списку с объёмами текстов равными 80% от исходных
for train_len_share in train_len_shares:

    # Запускаем "счётчик" номера текста
    t_num += 1

    # Выводим на экран сообщение о номере и объёме текста в 80% от исходного
    print(f'Доля 80% от текста №{t_num}: {train_len_share} символов')

```

Доля 80% от текста №1: 107628 символов  
Доля 80% от текста №2: 170705 символов

Рисунок 37 – Вычисление объёма 80% от каждого текста

```

✓from itertools import chain
import numpy as np
from sklearn.model_selection import train_test_split

print("Исходные данные:")
print(f"  Негативных отзывов (символов): {len(texts_list[0])}")
print(f"  Позитивных отзывов (символов): {len(texts_list[1])}")

CHUNK_SIZE = 500
STEP = 250

✓def split_text_into_chunks(text, chunk_size, step):
    """Разбивает текст на перекрывающиеся фрагменты"""
    chunks = []
    ✓for i in range(0, len(text) - chunk_size + 1, step):
        |     chunks.append(text[i:i + chunk_size])
    |     return chunks

train_ratio = 0.8

neg_len = len(texts_list[0])
pos_len = len(texts_list[1])

neg_train_text = texts_list[0][:int(neg_len * train_ratio)]
neg_test_text = texts_list[0][int(neg_len * train_ratio):]

pos_train_text = texts_list[1][:int(pos_len * train_ratio)]
pos_test_text = texts_list[1][int(pos_len * train_ratio):]

print(f"\nРазмеры частей:")
print(f"  Негативный train: {len(neg_train_text)} символов")
print(f"  Негативный test: {len(neg_test_text)} символов")
print(f"  Позитивный train: {len(pos_train_text)} символов")
print(f"  Позитивный test: {len(pos_test_text)} символов")

neg_train_chunks = split_text_into_chunks(neg_train_text, CHUNK_SIZE, STEP)
neg_test_chunks = split_text_into_chunks(neg_test_text, CHUNK_SIZE, STEP)

pos_train_chunks = split_text_into_chunks(pos_train_text, CHUNK_SIZE, STEP)
pos_test_chunks = split_text_into_chunks(pos_test_text, CHUNK_SIZE, STEP)

```

Рисунок 38 – Балансировка выборки (разбиение на фрагменты, train/test)

```

from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(max_features=5000,
                             ngram_range=(1, 2),
                             min_df=2,
                             max_df=0.8)
X_train_bow = vectorizer.fit_transform(X_train_texts)
X_test_bow = vectorizer.transform(X_test_texts)

X_train_bow = X_train_bow.toarray()
X_test_bow = X_test_bow.toarray()

y_train = np.array(y_train)
y_test = np.array(y_test)

print(f"Форма BoW обучающей выборки: {X_train_bow.shape}")
print(f"Форма BoW тестовой выборки: {X_test_bow.shape}")
print(f"Размер словаря: {len(vectorizer.get_feature_names_out())}")

```

Форма BoW обучающей выборки: (1110, 5000)  
 Форма BoW тестовой выборки: (275, 5000)  
 Размер словаря: 5000

Рисунок 39 – Bag of Words (BoW) векторизация текстов

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.utils.class_weight import compute_class_weight

class_weights = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weight_dict = {0: class_weights[0], 1: class_weights[1]}
print(f"Веса классов: {class_weight_dict}")

model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(X_train_bow.shape[1],)))
model.add(Dropout(0.5))
model.add(BatchNormalization())

model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(BatchNormalization())

model.add(Dense(32, activation='relu'))
model.add(Dropout(0.4))

model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True, mode='max')

history = model.fit(
    X_train_bow, y_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_test_bow, y_test),
    callbacks=[early_stop],
    class_weight=class_weight_dict,
    verbose=1
)

```

Рисунок 40 – Создание и обучение полносвязной нейросети на BoW

```

Веса классов: {0: np.float64(1.2937062937062938), 1: np.float64(0.8149779735682819)}
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
35/35 ————— 9s 91ms/step - accuracy: 0.5748 - loss: 0.8193 - val_accuracy: 0.5491 - val_loss: 0.6866
Epoch 2/50
35/35 ————— 0s 6ms/step - accuracy: 0.6234 - loss: 0.6628 - val_accuracy: 0.6473 - val_loss: 0.6655
Epoch 3/50
35/35 ————— 0s 6ms/step - accuracy: 0.7252 - loss: 0.5364 - val_accuracy: 0.7382 - val_loss: 0.6182
Epoch 4/50
35/35 ————— 0s 6ms/step - accuracy: 0.8099 - loss: 0.4147 - val_accuracy: 0.8036 - val_loss: 0.5390
Epoch 5/50
35/35 ————— 0s 6ms/step - accuracy: 0.8441 - loss: 0.3210 - val_accuracy: 0.8327 - val_loss: 0.4554
Epoch 6/50
35/35 ————— 0s 6ms/step - accuracy: 0.9216 - loss: 0.2115 - val_accuracy: 0.8400 - val_loss: 0.3987
Epoch 7/50
35/35 ————— 0s 6ms/step - accuracy: 0.9523 - loss: 0.1557 - val_accuracy: 0.8400 - val_loss: 0.3637
Epoch 8/50
35/35 ————— 0s 6ms/step - accuracy: 0.9550 - loss: 0.1264 - val_accuracy: 0.8436 - val_loss: 0.3526
Epoch 9/50
35/35 ————— 0s 6ms/step - accuracy: 0.9622 - loss: 0.1069 - val_accuracy: 0.8400 - val_loss: 0.3537
Epoch 10/50
35/35 ————— 0s 6ms/step - accuracy: 0.9739 - loss: 0.0742 - val_accuracy: 0.8436 - val_loss: 0.3475
Epoch 11/50
35/35 ————— 0s 6ms/step - accuracy: 0.9946 - loss: 0.0426 - val_accuracy: 0.8473 - val_loss: 0.3606
Epoch 12/50
35/35 ————— 0s 5ms/step - accuracy: 0.9892 - loss: 0.0436 - val_accuracy: 0.8436 - val_loss: 0.3646
Epoch 13/50
...
Epoch 15/50
35/35 ————— 0s 6ms/step - accuracy: 0.9946 - loss: 0.0289 - val_accuracy: 0.8400 - val_loss: 0.4050
Epoch 16/50
35/35 ————— 0s 6ms/step - accuracy: 0.9955 - loss: 0.0222 - val_accuracy: 0.8364 - val_loss: 0.4284
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Рисунок 41 – Процесс обучения

```

# Оценка на тестовой выборке
test_loss, test_acc = model.evaluate(X_test_bow, y_test, verbose=0)

print(f"Точность на тестовой выборке: {test_acc*100:.2f}%")

```

Точность на тестовой выборке: 84.73%

Рисунок 42 – Оценка точности на тестовой выборке

### Задание 3.

В домашнем задании Ultra Pro занятия по обработке текстов с помощью НС мы ставим задачу распознать уже не 6, как ранее, а целых 20 русских писателей! Это подразумевает и больший размер базы для обучения соответственно. Ячейка для скачивания базы уже включена в ноутбук задания.

В задании необходимо выполнить следующие пункты:

1. Загрузить саму базу по ссылке и подготовить файлы базы для обработки.
2. Создать обучающую и проверочную выборки, обратив особое внимание на балансировку базы: количество примеров каждого класса должно быть примерно одного порядка. При этом для разбивки необходимо применить цикл. Проверочная выборка должна быть 20% от общей выборки.
3. Подготовьте выборки для обучения и обучите сеть. Добейтесь результата точности сети не менее 95% на проверочной выборке модели Bag of Words и 75-80% – для модели Embedding.

```
import gdown
from zipfile import ZipFile
import os

# Скачивание архива
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/20writers.zip', None, quiet=True)

# Распаковка
with ZipFile("20writers.zip", 'r') as zip_ref:
    zip_ref.extractall("20writers")
```

Рисунок 43 – Скачивание и распаковка архива с текстами 20 писателей

```

import os
import re
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Embedding, LSTM, Bidirectional
from tensorflow.keras.callbacks import EarlyStopping

%matplotlib inline

```

Рисунок 44 – Импорт библиотек

```

def load_writers_data(data_dir):
    texts = []
    class_names = []
    files = os.listdir(data_dir)
    files.sort()
    for fname in files:
        if fname.endswith(".txt"):
            author = fname.replace(".txt", "")
            class_names.append(author)
            with open(os.path.join(data_dir, fname), 'r', encoding='utf-8') as f:
                text = f.read().replace('\n', ' ')
            texts.append(text)
    return texts, class_names

data_dir = "20writers"
texts, class_names = load_writers_data(data_dir)

print(f"Загружено файлов: {len(texts)}")
print(f"Классы: {class_names[:5]}...")
print(f"Всего классов: {len(class_names)}")

```

Загружено файлов: 20  
 Классы: ['Беляев', 'Булгаков', 'Васильев', 'Гоголь', 'Гончаров']...  
 Всего классов: 20

Рисунок 45 – Загрузка текстов 20 писателей и формирование списка классов

```

CHUNK_SIZE = 1000
STEP = 500

def split_text_into_chunks(text, chunk_size, step):
    chunks = []
    for start in range(0, len(text) - chunk_size + 1, step):
        chunks.append(text[start:start + chunk_size])
    return chunks

X_chunks = []
y_chunks = []

for class_id, full_text in enumerate(texts):
    chunks = split_text_into_chunks(full_text, CHUNK_SIZE, STEP)
    X_chunks.extend(chunks)
    y_chunks.extend([class_id] * len(chunks))
    print(f"{class_names[class_id]:20s} → {len(chunks)} фрагментов")

print(f"\nВсего фрагментов: {len(X_chunks)}")
print(f"Классов: {len(set(y_chunks))}")

```

|             |                    |
|-------------|--------------------|
| Беляев      | → 4509 фрагментов  |
| Булгаков    | → 4001 фрагментов  |
| Васильев    | → 5904 фрагментов  |
| Гоголь      | → 3929 фрагментов  |
| Гончаров    | → 6208 фрагментов  |
| Горький     | → 5045 фрагментов  |
| Грибоедов   | → 1937 фрагментов  |
| Достоевский | → 10356 фрагментов |
| Каверин     | → 3984 фрагментов  |
| Катаев      | → 5157 фрагментов  |
| Куприн      | → 4656 фрагментов  |
| Лермонтов   | → 3944 фрагментов  |
| Лесков      | → 4303 фрагментов  |
| Носов       | → 4797 фрагментов  |
| Пастернак   | → 6177 фрагментов  |
| Пушкин      | → 6815 фрагментов  |
| Толстой     | → 6713 фрагментов  |
| Тургенев    | → 3959 фрагментов  |
| Чехов       | → 13222 фрагментов |
| Шолохов     | → 6771 фрагментов  |

Всего фрагментов: 112387  
Классов: 20

Рисунок 46 – а Балансировка выборки: разбиение каждого текста на фрагменты (цикл по классам)

```
# Разделение
X_train_texts, X_test_texts, y_train, y_test = train_test_split(
    X_chunks, y_chunks,
    test_size=0.2,
    random_state=42,
    stratify=y_chunks
)

# Преобразуем в numpy
y_train = np.array(y_train)
y_test = np.array(y_test)

print(f"Обучающая выборка: {len(X_train_texts)} фрагментов")
print(f"Тестовая выборка: {len(X_test_texts)} фрагментов")
print(f"Классов в train: {len(np.unique(y_train))}")
print(f"Классов в test: {len(np.unique(y_test))}")
```

```
Обучающая выборка: 89909 фрагментов
Тестовая выборка: 22478 фрагментов
Классов в train: 20
Классов в test: 20
```

Рисунок 47 – Разделение на обучающую (80%) и тестовую (20%) выборки

```

MAX_BOW_FEATURES = 4000

vectorizer = CountVectorizer(
    max_features=MAX_BOW_FEATURES,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.8,
    stop_words=None
)
X_train_bow = vectorizer.fit_transform(X_train_texts)
X_test_bow = vectorizer.transform(X_test_texts)

X_train_bow = X_train_bow.toarray()
X_test_bow = X_test_bow.toarray()

gc.collect()

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping

model_bow = Sequential()
model_bow.add(Dense(256, activation='relu', input_shape=(X_train_bow.shape[1],)))
model_bow.add(Dropout(0.4))
model_bow.add(BatchNormalization())
model_bow.add(Dense(128, activation='relu'))
model_bow.add(Dropout(0.4))
model_bow.add(BatchNormalization())
model_bow.add(Dense(64, activation='relu'))
model_bow.add(Dropout(0.3))
model_bow.add(Dense(len(class_names), activation='softmax'))

model_bow.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True, mode='max')

history_bow = model_bow.fit(
    X_train_bow, y_train,
    epochs=30,
    batch_size=64,
    validation_data=(X_test_bow, y_test),
    callbacks=[early_stop],
    verbose=1
)

```

Рисунок 48 – Bag of Words векторизация и обучение нейросети

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape` to
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/30
1405/1405 ————— 18s 9ms/step - accuracy: 0.5131 - loss: 1.6585 - val_accuracy: 0.8039 - val_loss: 0.6740
Epoch 2/30
1405/1405 ————— 6s 5ms/step - accuracy: 0.7509 - loss: 0.8448 - val_accuracy: 0.8525 - val_loss: 0.5019
Epoch 3/30
1405/1405 ————— 6s 5ms/step - accuracy: 0.8122 - loss: 0.6402 - val_accuracy: 0.8690 - val_loss: 0.4316
Epoch 4/30
1405/1405 ————— 6s 4ms/step - accuracy: 0.8477 - loss: 0.5179 - val_accuracy: 0.8834 - val_loss: 0.3844
Epoch 5/30
1405/1405 ————— 10s 4ms/step - accuracy: 0.8688 - loss: 0.4464 - val_accuracy: 0.8872 - val_loss: 0.3679
Epoch 6/30
1405/1405 ————— 6s 5ms/step - accuracy: 0.8827 - loss: 0.3959 - val_accuracy: 0.8913 - val_loss: 0.3562
Epoch 7/30
1405/1405 ————— 6s 4ms/step - accuracy: 0.8948 - loss: 0.3561 - val_accuracy: 0.8959 - val_loss: 0.3381
Epoch 8/30
1405/1405 ————— 7s 5ms/step - accuracy: 0.9044 - loss: 0.3214 - val_accuracy: 0.8952 - val_loss: 0.3358
Epoch 9/30
1405/1405 ————— 6s 4ms/step - accuracy: 0.9119 - loss: 0.2934 - val_accuracy: 0.9008 - val_loss: 0.3205
Epoch 10/30
1405/1405 ————— 7s 5ms/step - accuracy: 0.9195 - loss: 0.2708 - val_accuracy: 0.9003 - val_loss: 0.3244
Epoch 11/30
1405/1405 ————— 6s 4ms/step - accuracy: 0.9237 - loss: 0.2569 - val_accuracy: 0.9043 - val_loss: 0.3136
Epoch 12/30
1405/1405 ————— 6s 5ms/step - accuracy: 0.9270 - loss: 0.2463 - val_accuracy: 0.9045 - val_loss: 0.3146
Epoch 13/30
...
Epoch 27/30
1405/1405 ————— 7s 5ms/step - accuracy: 0.9568 - loss: 0.1428 - val_accuracy: 0.9157 - val_loss: 0.2922
Epoch 28/30
1405/1405 ————— 6s 4ms/step - accuracy: 0.9579 - loss: 0.1404 - val_accuracy: 0.9157 - val_loss: 0.2901

```

Рисунок 49 – Обучение

```

MAX_WORDS = 15000
MAX_LEN = 200

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
tokenizer.fit_on_texts(X_train_texts)

X_train_seq = tokenizer.texts_to_sequences(X_train_texts)
X_test_seq = tokenizer.texts_to_sequences(X_test_texts)

X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post', truncating='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LEN, padding='post', truncating='post')

model_emb = Sequential()
model_emb.add(Embedding(MAX_WORDS, 64, input_length=MAX_LEN))
model_emb.add(Flatten())
model_emb.add(Dense(128, activation='relu'))
model_emb.add(Dropout(0.4))
model_emb.add(BatchNormalization())
model_emb.add(Dense(64, activation='relu'))
model_emb.add(Dropout(0.3))
model_emb.add(Dense(len(class_names), activation='softmax'))

model_emb.build(input_shape=(None, MAX_LEN))

model_emb.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model_emb.summary()

early_stop = EarlyStopping(monitor='val_accuracy', patience=4, restore_best_weights=True, mode='max')

history_emb = model_emb.fit(
    X_train_pad, y_train,
    epochs=20,
    batch_size=128,
    validation_data=(X_test_pad, y_test),
    callbacks=[early_stop],
    verbose=1
)

test_loss_emb, test_acc_emb = model_emb.evaluate(X_test_pad, y_test, verbose=0)

```

Рисунок 50 – Embedding + Flatten модель

| Layer (type)                               | Output Shape    | Param #   |
|--------------------------------------------|-----------------|-----------|
| embedding (Embedding)                      | (None, 200, 64) | 960,000   |
| flatten (Flatten)                          | (None, 12800)   | 0         |
| dense_4 (Dense)                            | (None, 128)     | 1,638,528 |
| dropout_3 (Dropout)                        | (None, 128)     | 0         |
| batch_normalization_2 (BatchNormalization) | (None, 128)     | 512       |
| dense_5 (Dense)                            | (None, 64)      | 8,256     |
| dropout_4 (Dropout)                        | (None, 64)      | 0         |
| dense_6 (Dense)                            | (None, 20)      | 1,300     |

**Total params: 2,608,596 (9.95 MB)**

**Trainable params: 2,608,340 (9.95 MB)**

**Non-trainable params: 256 (1.00 KB)**

Рисунок 51 – Структура модели

|             |         |              |                    |                |                        |                    |
|-------------|---------|--------------|--------------------|----------------|------------------------|--------------------|
| Epoch 1/20  | 703/703 | 10s 9ms/step | - accuracy: 0.3579 | - loss: 2.0713 | - val_accuracy: 0.6719 | - val_loss: 0.9971 |
| Epoch 2/20  | 703/703 | 3s 4ms/step  | - accuracy: 0.7303 | - loss: 0.8361 | - val_accuracy: 0.8030 | - val_loss: 0.6237 |
| Epoch 3/20  | 703/703 | 3s 4ms/step  | - accuracy: 0.8638 | - loss: 0.4263 | - val_accuracy: 0.8017 | - val_loss: 0.7044 |
| Epoch 4/20  | 703/703 | 4s 5ms/step  | - accuracy: 0.9201 | - loss: 0.2506 | - val_accuracy: 0.8559 | - val_loss: 0.5204 |
| Epoch 5/20  | 703/703 | 3s 5ms/step  | - accuracy: 0.9460 | - loss: 0.1703 | - val_accuracy: 0.8572 | - val_loss: 0.5100 |
| Epoch 6/20  | 703/703 | 3s 4ms/step  | - accuracy: 0.9580 | - loss: 0.1336 | - val_accuracy: 0.8547 | - val_loss: 0.5340 |
| Epoch 7/20  | 703/703 | 3s 4ms/step  | - accuracy: 0.9652 | - loss: 0.1107 | - val_accuracy: 0.8672 | - val_loss: 0.5158 |
| Epoch 8/20  | 703/703 | 4s 5ms/step  | - accuracy: 0.9714 | - loss: 0.0927 | - val_accuracy: 0.8767 | - val_loss: 0.5051 |
| Epoch 9/20  | 703/703 | 3s 4ms/step  | - accuracy: 0.9744 | - loss: 0.0814 | - val_accuracy: 0.8596 | - val_loss: 0.5988 |
| Epoch 10/20 | 703/703 | 3s 4ms/step  | - accuracy: 0.9752 | - loss: 0.0782 | - val_accuracy: 0.8620 | - val_loss: 0.5832 |
| Epoch 11/20 | 703/703 | 3s 4ms/step  | - accuracy: 0.9788 | - loss: 0.0671 | - val_accuracy: 0.8721 | - val_loss: 0.5334 |
| Epoch 12/20 | 703/703 | 4s 5ms/step  | - accuracy: 0.9804 | - loss: 0.0632 | - val_accuracy: 0.8755 | - val_loss: 0.5410 |

Рисунок 52 – Обучение

```
print(f"Bag of Words точность:      {test_acc_bow:.2%}")
print(f"Embedding точность:         {test_acc_emb:.2%}")
```

|                        |        |
|------------------------|--------|
| Bag of Words точность: | 91.64% |
| Embedding точность:    | 87.67% |

Рисунок 53 – Итоговая точность моделей

**Вывод:** в ходе лабораторной работы были изучены основные подходы к обработке и классификации текстовых данных с использованием нейронных сетей. Освоены методы токенизации, преобразования текста в числовые последовательности, а также техники векторизации Bag of Words и Embedding. Реализованы модели полносвязных нейросетей со слоями регуляризации Dropout и BatchNormalization для задач многоклассовой классификации авторов и бинарной классификации тональности отзывов. Проведены эксперименты по подбору гиперпараметров (размер окна, шаг окна, размер словаря), результаты которых позволили выявить оптимальные конфигурации для каждого типа задач. Для классификации 6 писателей лучшая точность составила 82,48% на модели с размером окна 2000 и шагом 100. В задаче бинарной классификации отзывов об автомобиле Tesla достигнута точность 84,73%, что соответствует требованиям задания. При распознавании 20 русских писателей с использованием подхода Bag of Words получена точность 91,64% (с учётом ограничений оперативной памяти Colab), а модель на основе Embedding показала результат 87,67%, превысив требуемый диапазон 75–80%.